



02 Frontend Dinámico con TypeScript y Vite

06 Consumo de APIs Asíncronas con Fetch y Lanzamiento
del Hito 2

06 Consumo de APIs Asíncronas con Fetch y Lanzamiento del Hito 2

- Unidad 1: Fundamentos de Calidad y TDD en Java
- Unidad 2: Frontend Dinámico con TypeScript y Vite
- Unidad 3: Arquitectura Limpia y Diseño Guiado por el Dominio (DDD)
- Unidad 4: Microservicios con Spring Boot, PostgreSQL y Docker
- Unidad 5: Potenciación con IA y Prompting Técnico
- Unidad 6: Integración Full-Stack y Validación



*Te encuentras
aquí*



¿Qué aprenderás en esta sesión?

- *Implementar llamadas de red asíncronas utilizando la API nativa Fetch encapsuladas en bloques estructurados de control de errores try/catch.*
- *Mapear las respuestas JSON externas directamente a interfaces fuertemente tipadas de TypeScript.*
- *Consolidar el desarrollo frontend independiente para cumplir con los criterios de evaluación del Hito 2.*

Contexto



Contexto: La Conexión de la Cartelera de Conciertos Real

Hasta el momento, nuestra cartelera de conciertos en NeonPulse funciona utilizando un arreglo de datos simulado escrito en duro (mock data) dentro de nuestros archivos locales. Esto significa que si una banda independiente actualiza el precio de su entrada o añade una nueva fecha desde el backend, nuestros usuarios jamás verán el cambio a menos que volvamos a compilar y subir todo el frontend.

Para que la aplicación sea dinámica, debemos conectar los componentes con endpoints del servidor. Dado que la comunicación de red está sujeta a latencias y fallos, aprenderemos a consumir datos remotos con Fetch y a gestionar excepciones para mantener la estabilidad de la interfaz.

/* i. El “Arte” de Consumir Datos Remotos con la API Fetch */

i. El “Arte” de Consumir Datos Remotos con la API Fetch

Integración Asíncrona con la API Fetch

Fetch es la interfaz nativa del navegador para realizar peticiones HTTP asíncronas basadas en promesas. Su estructura se blindo mediante bloques *try / catch* para capturar anomalías de red sin romper la aplicación.

Las Dos Fases del Desempaquetado Asíncrono:

1. **Validación de Canal (*fetch*)**: La instrucción `await fetch(url)` se resuelve al recibir las cabeceras. Es obligatorio validar la propiedad booleana `response.ok`. Si es falsa (errores 404 o 500), se debe forzar la salida arrojando un error explícito.
2. **Conversión de Payload (*.json()*)**: Si el canal es válido, `await response.json()` transforma el flujo de texto plano en objetos interactivos.

Buenas Prácticas

En la fase de conversión es donde se aplica el tipado estricto de TypeScript, asignando la interfaz correspondiente al resultado para activar el autocompletado seguro en toda la arquitectura.

Ejercicios



Ejercicio 1

Consumo Asíncrono de la Cartelera de Bandas

Objetivo de Ingeniería

Reemplazar los datos fijos locales de NeonPulse por una llamada asíncrona a un endpoint externo simulado, controlando los estados de carga en el DOM.

Vamos al código...



Cierre de sesión

Conceptos clave

- Mapeo Estricto.
- Robustez con Catch.
- Feedback Continua.



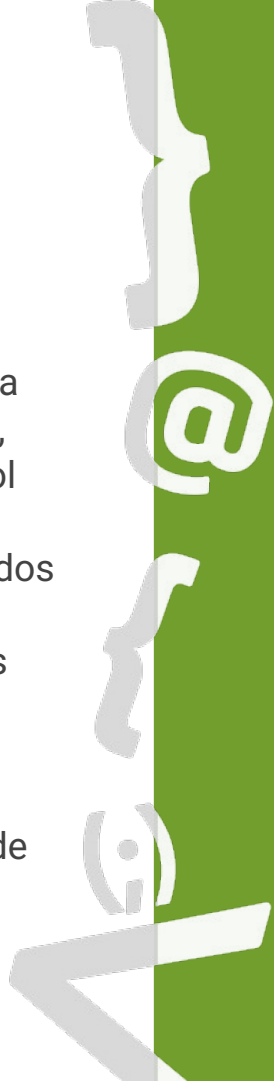
Hito 2



Hito 2

El entregable formal consiste en la publicación de la base del proyecto frontend en GitHub, asegurando el cumplimiento de los siguientes tres requerimientos:

1. **Modelado y Tipado de Estructuras** (3 Puntos): Los archivos .ts deben modelar de forma hermética las entidades de su negocio utilizando interfaces y enumeraciones estrictas, bloqueando el uso de variables genéricas (any) o cadenas de texto libres para el control de estados.
2. **Renderizado Seguro y Gestión de Eventos** (3 Puntos): La interfaz debe capturar los nodos del DOM aplicando guardias de tipo para prevenir referencias nulas, neutralizar el comportamiento por defecto de los formularios mediante preventDefault() y extraer los datos de forma limpia aplicando aserciones especializadas.
3. **Simulación Asíncrona con Bloques de Control** (4 Puntos): Implementar funciones asíncronas preparadas con la sintaxis async/await y bloques try/catch para simular llamadas a servicios externos de carga y envío de datos, inyectando estados visuales de carga en el DOM para guiar la interacción.



Hito 2

Criterio Evaluado	3-4 Puntos (Excelente)	2 Puntos (Competente)	1 Punto (Insuficiente)	0 Puntos (No Entrega)
Modelado de Datos en TS	Las entidades y estados lógicos están completamente modelados con interfaces y enumstrictos. Cero uso de any.	Modela los datos, pero utiliza cadenas libres en lugar de enumeraciones para controlar los estados críticos de las variables.	Estructuras de datos desorganizadas; se recurre al tipado dinámico eludiendo la compilación estricta de TypeScript.	Ausencia total de tipos o modelos estructurados en los archivos del repositorio.
Manejo del DOM y Formularios	Captura segura de nodos con validación de nulidad. Neutraliza formularios con preventDefault() y extrae payloads limpios.	Captura elementos y gentiona eventos, pero carece de guardias de tipo contra valores nulos o mezcla lógicas de marcado.	Los formularios recargan la página nativamente o causan excepciones de tipo no controladas en la consola del navegador.	Código ausente o incapaz de capturar interacciones básicas de usuario en la pantalla.
Arquitectura Asíncrona (4 Pts)	(4 Puntos): Estructura funciones con async/await y try/catchimpecables, controlando de manera visual los estados de carga y errores en el DOM.	(2 Puntos):Implementa llamadas de red, pero carece de bloques catch para mitigar fallos físicos o no inyecta feedback de carga en la pantalla.	(1 Punto): Uso incorrecto de la asincronía que provoca bloqueos en la interfaz o procesamiento de datos corruptos sin controles lógicos.	(0 Puntos): No se implementan lógicas asíncronas ni consumos de datos externos en el desarrollo.



{desafío}
latam_

*Academia de
talentos digitales*

